TypeScript: From JavaScript to Type Safety

Programming Languages / Web Development

What is TypeScript?

- An open-source language developed by Microsoft
- A typed superset of JavaScript
- Adds static analysis and developer tooling
- Compiles (transpiles) to JavaScript
- Existing JavaScript code is valid TypeScript

Why do we need it?

- JavaScript is dynamically typed
- Some errors appear only during execution
- Large codebases become harder to maintain
- Types document intent and improve autocomplete

```
function total(price, qty) {
  return price * qty;
}
total("ten", 2); // NaN
```

JavaScript vs TypeScript

- JavaScript: types are checked at runtime
- TypeScript: many errors are caught before running
- Type annotations disappear in generated JS
- TypeScript does not change JS runtime semantics

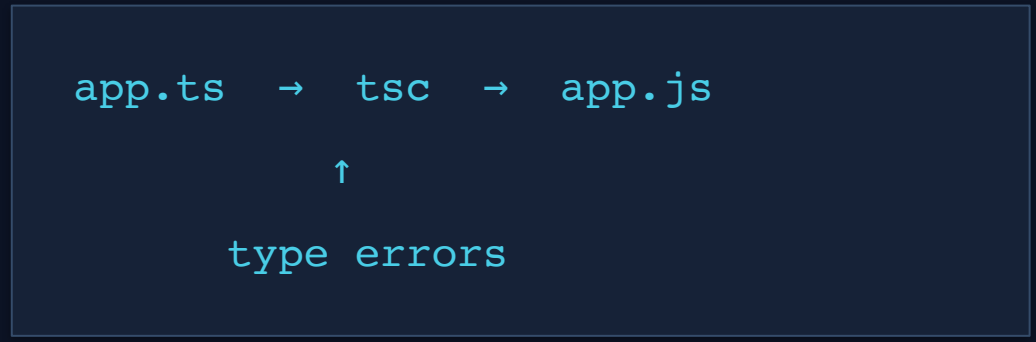
```
// JavaScript
let score = "high";
score = 100;

// TypeScript
let score: string = "high";
score = 100; // Type error
```

How TypeScript works

- Write source files ending in .ts or .tsx

- Compiler checks types and reports diagnostics
- Compiler emits JavaScript (depending on settings)
- JavaScript executes in browser or Node.js



Set up a project

- Install Node.js and npm
- Initialize a project and install TypeScript
- Create a tsconfig.json configuration
- Compile with npx tsc

```
npm init -y
npm install --save-dev typescript
npx tsc --init
npx tsc
```

Primitive types

- string: text values
- number: integers and floating point
- boolean: true or false
- null and undefined: absent values
- bigint and symbol: specialized primitives

```
let name: string = "Maya";
let year: number = 2026;
let enrolled: boolean = true;
```

Type inference

- Annotations are often optional
- The compiler infers types from initial values
- Reassignments must respect inferred types
- Explicit types help at API boundaries

```
let credits = 30; // number
credits = 36; // OK
credits = "thirty"; // Error
```

Arrays, tuples and readonly

- Arrays hold values of a specified type
- Tuples model fixed positions and types
- readonly helps prevent accidental mutation

```
const grades: number[] = [8, 9, 10];

const student: [string, number] = ["Ana", 20];

const ids: readonly number[] = [1, 2];
```

Objects and interfaces

- Object types describe required properties
- Interfaces name reusable object contracts
- Optional properties use ?
- readonly properties cannot be reassigned

```
interface Student {
  readonly id: number;
  name: string;
  email?: string;
}
const s: Student = { id: 1, name:
"Ana" };
```

Functions and return types

- Annotate parameters for reliable calls
- Return types may be inferred or explicit
- Optional/default parameters express flexibility
- Callbacks can be typed

```
function average(a: number, b:
number): number {
  return (a + b) / 2;
}
average(8, 10); // 9
```

Union and literal types

- A union allows more than one type
- Literal types restrict values to exact choices
- Useful for statuses and API responses

```
type ID = string | number;

type Status = "pending" | "done" |
"failed";

let status: Status = "pending";
```

Type narrowing

- Use typeof to distinguish primitive types
- Use in, instanceof and discriminants for objects
- Control flow narrows possible types

```
function format(value: string | number) {
  if (typeof value === "number") {
    return value.toFixed(2);
  }
  return value.toUpperCase();
}
```

Type aliases vs interfaces

- Both can describe object shapes
- Aliases also represent unions, tuples and primitives
- Interfaces support declaration merging
- Choose consistent conventions for your team

```
type Result = "ok" | "error";
interface User {
  id: number;
  username: string;
}
```

Generics: reusable types

- Generics preserve information about input types
- Avoid duplicating functions for each type
- Common in collections, APIs and libraries

```
function first<T>(items: T[]): T | undefined {
  return items[0];
}
const firstName = first(["Ada", "Lin"]);
```

Classes and access modifiers

- Classes support typed fields and methods
- public, private and protected control access
- implements checks a class against an interface

```
class Course {
  constructor(public title: string,
              private seats:
number) {}
  hasSpace(): boolean {
    return this.seats > 0;
  }
}
```

Special types: any, unknown, never

- any disables most checking: avoid when possible
- unknown requires narrowing before use

- never represents impossible values or no return
- Prefer precise types over broad escape hatches

```
function parse(value: unknown) {
  if (typeof value === "string") {
    return value.trim();
  }
  return "invalid";
}
```

Working with APIs

- External JSON is not automatically trustworthy
- Define expected response types
- Validate untrusted data at runtime
- Static types do not replace input validation

```
interface ApiUser { id: number; name: string; }

// Type annotations alone cannot validate JSON.

// Validate before treating data as ApiUser.
```

Compiler configuration

- strict enables stronger checking
- target chooses JavaScript language level
- module selects module output behavior
- noEmit can check types without output

```
{
  "compilerOptions": {
    "strict": true,
    "target": "ES2022",
    "module": "NodeNext",
    "noEmit": true
  }
}
```

Where TypeScript is used

- Frontend apps: React, Angular and Vue
- Backend services: Node.js
- Shared types between client and server
- Large team projects and public libraries
- Developer tools and automation scripts

Benefits and trade-offs

- Benefits: earlier feedback, clearer APIs, safer refactoring
- Benefits: editor completion and maintainability
- Costs: learning curve and configuration
- Costs: types can be complex; compilation adds a step
- Runtime validation is still necessary

Mini exercise: find the bug

- What error will TypeScript report?
- How would you correct the function?
- Bonus: make the input type reusable

```
function welcome(name: string) {
  return "Welcome, " + name;
}
welcome(42);
```

Exercise solution

- The argument 42 is a number, not a string
- Fix by passing a string, or intentionally accepting both
- Types make the contract visible to callers

```
welcome("42");

// or

function welcome2(name: string | number) {
  return "Welcome, " + name;
}
```

Key takeaways

- TypeScript adds static types to JavaScript
- Types help catch mistakes before execution
- Interfaces, unions and generics model real applications
- TypeScript compiles to ordinary JavaScript
- Good types complement—not replace—runtime tests

References and further reading

Explore the Playground to edit and run the examples

- Official handbook: typescriptlang.org/docs/handbook/intro.html
- TypeScript playground: typescriptlang.org/play
- TSConfig reference: typescriptlang.org/tsconfig
- Microsoft TypeScript repository: github.com/microsoft/TypeScript